

Manual Técnico — Compilador y Traductor Codex Latinus a Pig Latin

Proyecto: Práctica 1 — Compiladores 2

Nombre: Elmer Miguel — **Carnet:** 201931295

24 de agosto de 2026

1. Introducción y Objetivos del Sistema

El compilador **Codex Latinus** es un sistema de procesamiento de lenguajes formales diseñado para analizar, validar semánticamente y traducir código fuente escrito en el lenguaje de programación Codex Latinus hacia el dialecto Pig Latin.

1.1. Objetivos Técnicos

- **Análisis Léxico y Sintáctico Robusto:** Implementación de un reconocedor mediante ANTLR4 con manejo de errores léxicos y sintácticos sin detener abruptamente el flujo de la aplicación.
- **Construcción de Árbol de Sintaxis Abstracta (AST) Unificado:** Conversión del Árbol de Derivación Concreta (CST) a un AST desacoplado de la herramienta de parsing mediante un modelo genérico de nodo (`AstNode`).
- **Monitoreo de la Pila de Reconocimiento:** Registro secuencial de estados de la pila del parser (operaciones SHIFT, REDUCE, ACCEPT) para fines didácticos y de depuración interactiva.
- **Análisis Semántico en Dos Pasadas:** Verificación estricta de declaraciones, resolución de ámbitos anidados, chequeo de tipos con inferencia jerárquica implícita, detección de código inalcanzable y verificación de retornos en funciones `ratio`.
- **Traducción Dirigida por AST:** Recorrido recursivo del AST para emitir código fuente equivalente en Pig Latin, aplicando transformaciones morfológicas sobre identificadores y palabras clave, y traduciendo sentencias especiales de entrada/salida (`%OINK`, `%OINK_OINK`).
- **Interfaz Gráfica Integrada:** Entorno de desarrollo interactivo (IDE) en Swing con resaltado de sintaxis en tiempo real, visualización de errores con navegación hacia el código, renderizado gráfico del AST mediante Graphviz, tabla de símbolos y simulador de pila.

2. Tecnologías Utilizadas

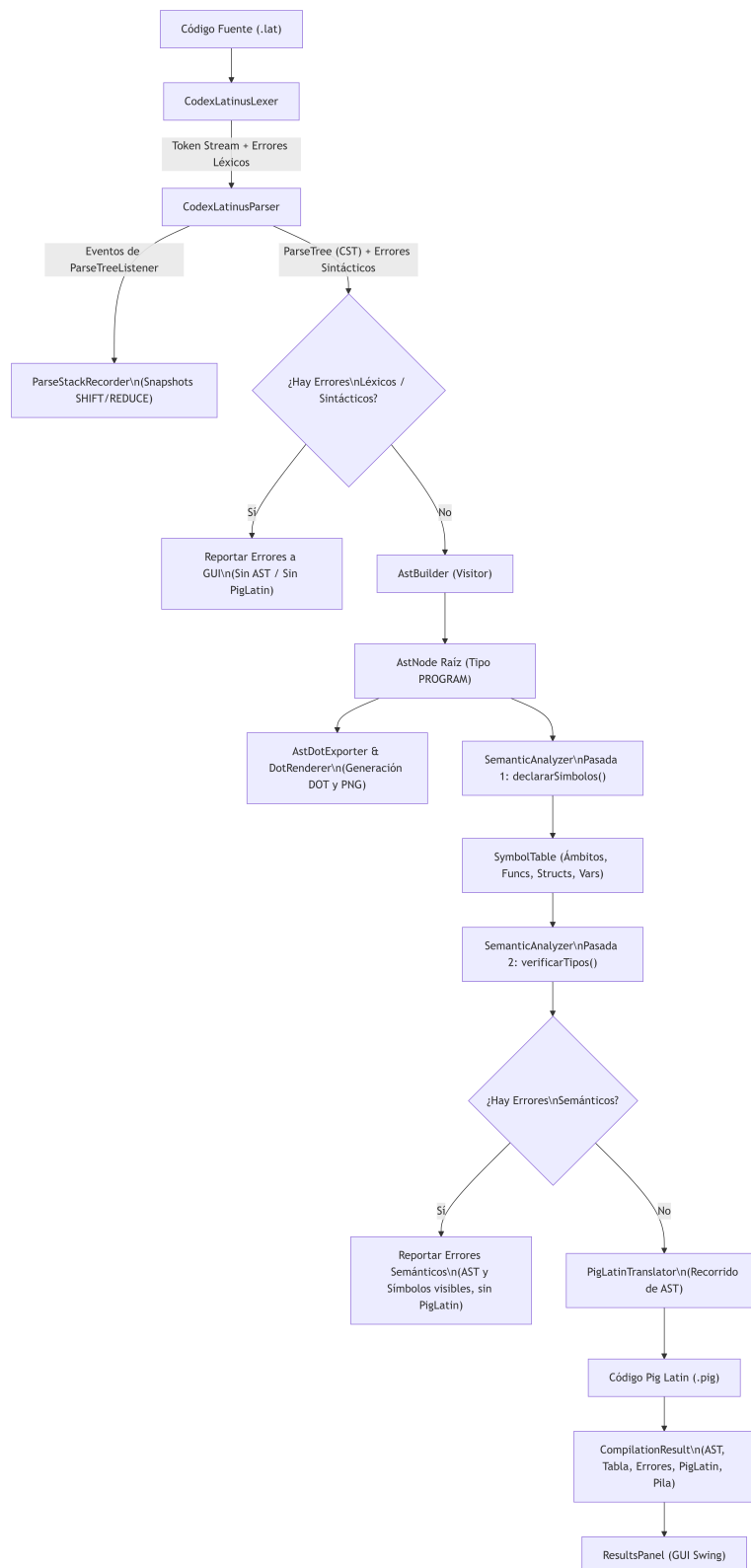
Componente / Herramienta	Versión	Propósito / Justificación
Java JDK	21 (LTS)	Lenguaje base. Permite el uso de switch exhaustivo con pattern matching, records, bloques de texto multilínea y colecciones inmutables.
ANTLR4 Runtime	4.9.2	Generación automática del lexer y parser predictivo LL(*) a partir de la especificación gramatical CodexLatinus.g4.
RSyntaxTextArea	4.0.1	Componente Swing para edición de texto con soporte de numeración de líneas, coloreado sintáctico personalizable y navegación.
Graphviz (dot)	2.40+ (Sistema)	Herramienta CLI externa invocada por DotRenderer para compilar archivos .dot en imágenes .png del AST en tiempo real.
Apache Maven	3.9+	Gestor de compilación, empaquetado y resolución de dependencias del ciclo de vida del proyecto.
Lombok	1.18.46	Procesamiento de anotaciones para reducción de código repetitivo en clases accesorias.
JTattoo	1.6.7	Biblioteca de Look & Feel para componentes visuales de la interfaz gráfica Swing.

Cuadro 1: Tecnologías y herramientas utilizadas en el proyecto.

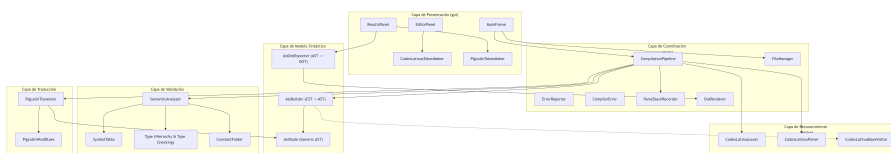
3. Arquitectura del Sistema

El sistema implementa un diseño modular por capas donde cada fase de la compilación opera de forma desacoplada y se comunica mediante estructuras de datos bien definidas coordinadas por la clase `CompilationPipeline`.

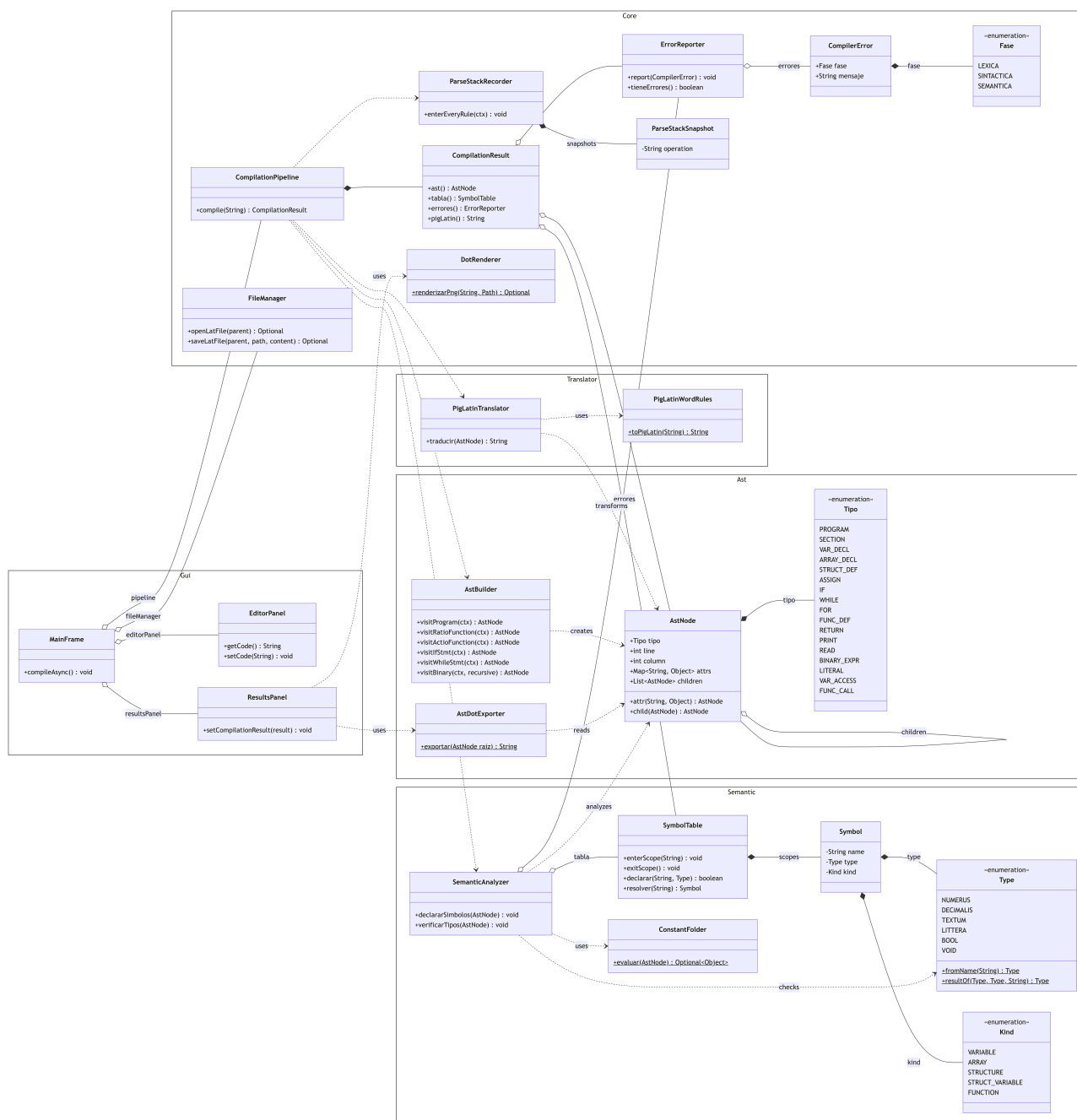
3.1. Diagrama de Pipeline de Compilación



3.2. Diagrama de Módulos y Paquetes



4. Diagrama de Clases del Sistema



5. Especificación del Lenguaje Codex Latinus

5.1. Estructura General del Programa

Un programa válido en Codex Latinus está estructurado en tres secciones delimitadas:

- **VARIABLES>** (Opcional): Declaraciones globales de variables, arreglos y estructuras.
- **MUNERA>** (Opcional): Declaraciones de funciones con o sin retorno (*ratio* / *actio*).
- **MAIOR>** (Obligatorio): Punto de entrada principal con instrucciones ejecutables y declaraciones locales.
- **FINIS**; (Obligatorio): Delimitador final de terminación del programa en mayúsculas.

5.2. Léxico y Tabla de Tokens

Categoría	Lexema / Patrón	Token ANTLR	Descripción
Secciones	VARIABLES> MUNERA> MAIOR> FINIS finis	SEC.VARIABLES SEC.MUNERA SEC.MAJOR FIN.PROGRAMA FIN.BLOQUE	Encabezado de sección de variables Encabezado de sección de funciones Encabezado de sección principal Cierre global del programa (con ;) Cierre de bloques (estructura, si, dum, funciones)
Palabras Clave	esto series estructura ratio actio reddere si aliter dum facere per perge interrumpe non	KW.ESTO KW.SERIES KW.STRUCTURA KW.RATIO KW.ACTIO KW.REDDERE KW.SI KW.ALITER KW.DUM KW.FACERE KW.PER KW.PERGE KW.INTERRUMPE NOT	Declaración de variable o parámetro Declaración de arreglos Definición de estructuras de datos Declaración de función con retorno de valor Declaración de función sin retorno (void) Sentencia de retorno de valor Sentencia condicional if Sentencia condicional else / else if Bucle while / condición de do-while Inicio de bucle do-while Bucle for con inicializador, condición y paso Sentencia continue Sentencia break Negación lógica unaria
Tipos Primitivos	numerus decimalis textum littera bool	KW.NUMERUS KW.DECIMALIS KW.TEXTUM KW.LITTERA KW.BOOL	Tipo entero Tipo decimal de punto flotante Tipo cadena de caracteres Tipo carácter individual Tipo lógico booleano explícito
Literales Bool	verum falsus	VERUM FALSUS	Literal verdadero (y forma corta de tipo) Literal falso (y forma corta de tipo)
Operadores E/S	>> <<	PRINT READ	Imprimir expresión por consola Leer entrada desde consola hacia variable
Operadores	+, -, *, / ==, !=, <, >, <=, >= &&, ++, -- =	PLUS, MINUS, STAR, SLASH EQ, NEQ, LT, GT, LE, GE AND, OR INC, DEC ASSIGN	Aritméticos estándar Relacionales y de igualdad Lógicos Incremento y decremento unitario Operador de asignación
Literales	[0-9]+ [0-9]+\.[0-9]+ "..." '...'	NUMERUS.LIT DECIMAL.LIT TEXTUM.LIT LITTERA.LIT	Enteros Decimales Cadenas delimitadas por comillas dobles Caracteres delimitados por comillas simples
Identificadores	[a-zA-Z][a-zA-Z0-9_]*	IDENT	Nombres de variables, funciones y estructuras
Comentarios	//... ##...##	LINE.COMMENT BLOCK.COMMENT	Comentario de una línea (descartado) Comentario multilinea de bloque (descartado)

Cuadro 2: Tabla de léxico y tokens del lenguaje Codex Latinus.

6. Diseño del Árbol de Sintaxis Abstracta (AST)

6.1. Patrón AstNode Genérico Unificado

Para optimizar el mantenimiento del compilador y evitar una jerarquía excesiva de clases concretas, se utiliza un modelo de nodo único `AstNode` parametrizado por el enum `Tipo`, con un mapa de atributos `attrs` (`LinkedHashMap<String, Object>`) y una lista de hijos `children` (`ArrayList<AstNode>`).

```
AstNode
+ tipo: AstNode.Tipo
+ line: int, column: int
+ attrs: LinkedHashMap<String, Object>
```

```

+ children: ArrayList<AstNode>
+ attr(name: String, value: Object): AstNode
+ child(node: AstNode): AstNode

```

6.2. Mapeo de Tipos de Nodos, Atributos e Hijos

Tipo de Nodo (Tipo)	Atributos en attrs	Estructura de children
PROGRAM	terminador ("FINIS")	[0] Globales, [1] Funciones, [2] Maior
SECTION	nombre ("VARIABLES", "MUNERA", "MAIOR"), delimitador	Lista de declaraciones o sentencias contenidas
VAR_DECL	nombre, tipoDeclarado	[0] Expresión inicial (opcional)
ARRAY_DECL	nombre, tipoElemento, tipoInferido (bool)	[0] Expr tamaño, [1] STRUCT.FIELD.INIT (init opcional)
STRUCT_DEF	nombre, cierre ("finis")	Lista de STRUCT.MEMBER
STRUCT_MEMBER	nombre, tipoDeclarado, esArreglo (bool)	Vacio
STRUCT_VAR_DECL	nombre, tipoDeclarado, conPuntoYComa	[0] STRUCT.FIELD.INIT (campos inicializados)
STRUCT_FIELD_INIT	nombreCampo (en campos individuales)	[0] Expr valor o sub-literal anidado
FUNC_DEF	nombre, tipoRetorno, esActio, paramCount, localCount, cierre	[0..paramCount-1] PARAM, luego SECTION locales y sentencias
PARAM	nombre, tipoDeclarado	Vacio
IF	cierre ("finis")	[0] Condición, [1] Bloque then, [2..N] Cláusulas else-if / else
WHILE	cierre ("finis")	[0] Condición, [1] Bloque cuerpo
DO_WHILE	cierre ()	[0] Bloque cuerpo, [1] Condición
FOR	cierre ()	[0] VAR_DECL (init), [1] Condición, [2] Update (ASSIGN/INC.DEC), [3] Bloque cuerpo
ASSIGN	esActualizacion (bool), conPuntoYComa	[0] VAR.ACCESS (lvalue), [1] Expr valor o struct literal
BINARY_EXPR	operador (+, -, *, /, ==, !=, i, <, >, <=, >=, &&,)	[0] Izquierda, [1] Derecha
UNARY_EXPR	operador (~, non)	[0] Operando
PAREN_EXPR	Vacio	[0] Expresión interna
LITERAL	valor (Object), tipoLiteral, raw	Vacio
VAR_ACCESS	nombreBase	Lista encadenada de pasos: ACCESS.STEP.PROP o ACCESS.STEP.INDEX
ACCESS_STEP_PROP	nombreCampo	Vacio
ACCESS_STEP_INDEX	Vacio	[0] Expresión de índice
FUNC_CALL	nombre, sentencia (bool)	Lista de expresiones de argumentos (argList)
INC_DEC	operador (++), sentencia (bool)	[0] VAR_ACCESS del identificador
PRINT	Vacio	Lista de expresiones a imprimir encadenadas con &&
READ	nombre (identificador destino opcional), conPuntoYComa	Vacio
RETURN	Vacio	[0] Expresión retornada
BREAK / CONTINUE	Vacio	Vacio

Cuadro 3: Mapeo de tipos de nodos, atributos e hijos del AST.

6.3. Ejemplo de Exportación DOT (Graphviz)

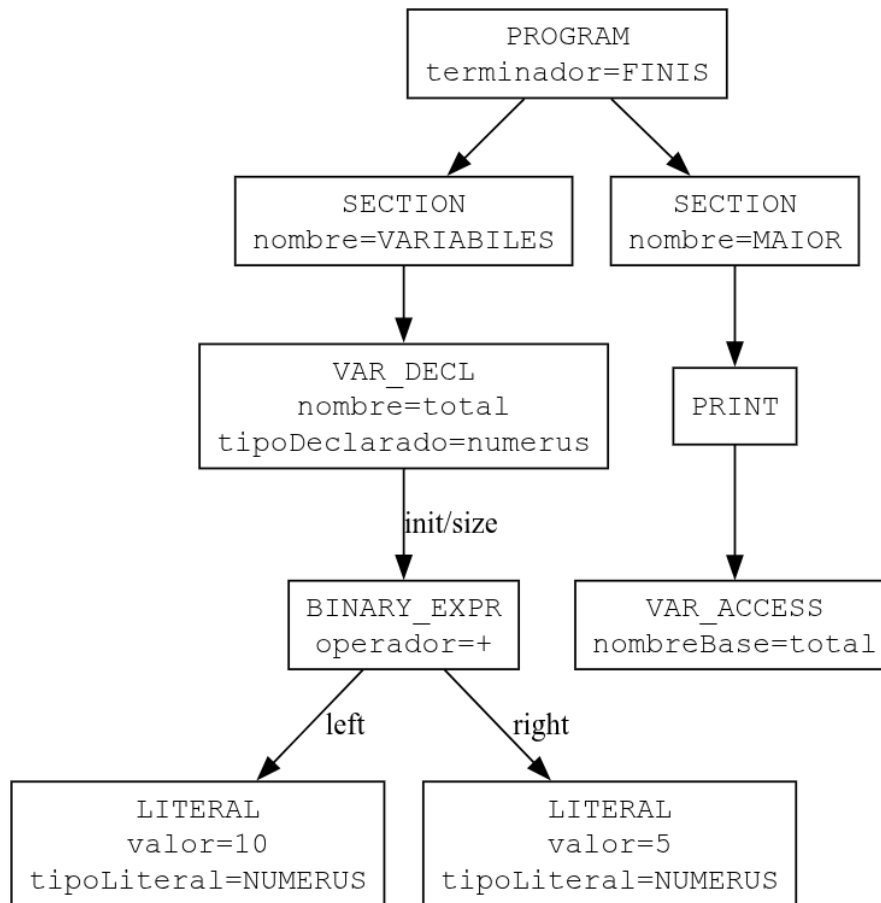
Dado el siguiente código fuente latino:

```

VARIABLES>
esto total : numerus 10 + 5;
MAIOR>
>> total;
FINIS;

```

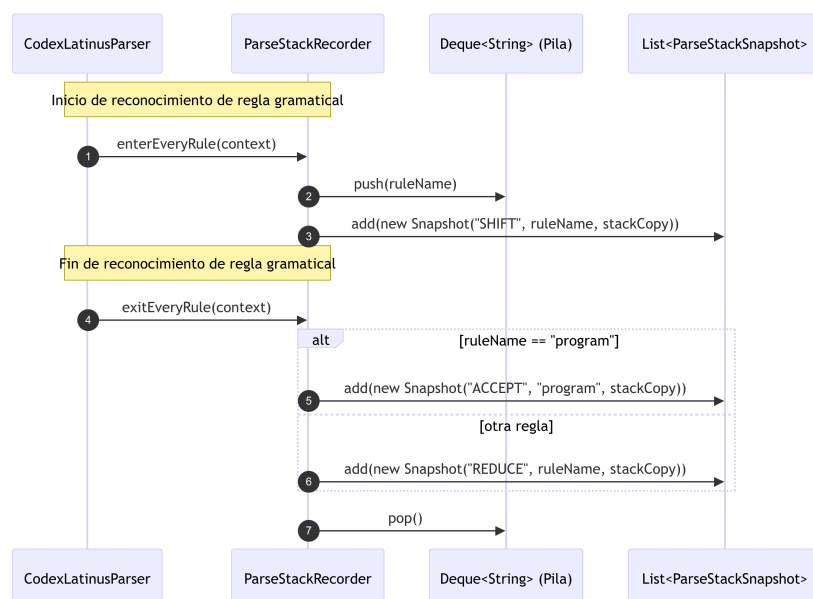
AstDotExporter genera el siguiente grafo DOT estándar:



7. Registro de la Pila de Reconocimiento Sintáctico

Debido a que ANTLR4 es un reconocedor predictivo LL(*) que opera mediante llamadas recursivas y no posee una tabla LR con pila explícita de transiciones de estados de autómatas, se diseñó la clase ParseStackRecorder que implementa ParseTreeListener para reconstruir didácticamente la pila sintáctica en tiempo de ejecución.

7.1. Mapeo de Operaciones de Pila



7.2. Funcionalidades en la Interfaz Gráfica

- **Navegación Paso a Paso:** Botones Atrás y Siguiente que permiten inspeccionar el estado de la pila en cualquier momento.
- **Modo Resumen Completo:** Botón Mostrar todo que lista la traza completa de operaciones SHIFT y REDUCE generadas durante el análisis.
- **Ventana Emergente Independiente:** Opción de abrir la pila en un JFrame maximizado para análisis detailed de trazas extensas.

8. Análisis Semántico y Sistema de Tipos

8.1. Jerarquía e Inferencia Implícita de Tipos

El sistema de tipos asigna un nivel de precedencia numérico a cada tipo primitivo:

- **Nivel 5:** textum (Cadena de texto)
- **Nivel 4:** decimalis (Punto flotante)
- **Nivel 3:** numerus (Entero)
- **Nivel 2:** littera (Carácter)
- **Nivel 1:** bool (verum / falsus) (Booleano)

Regla de Coerción: Un tipo destino T_d acepta un tipo fuente T_s mediante la relación:

$$\text{accepts}(T_s) \iff \text{precedence}(T_s) \leq \text{precedence}(T_d)$$

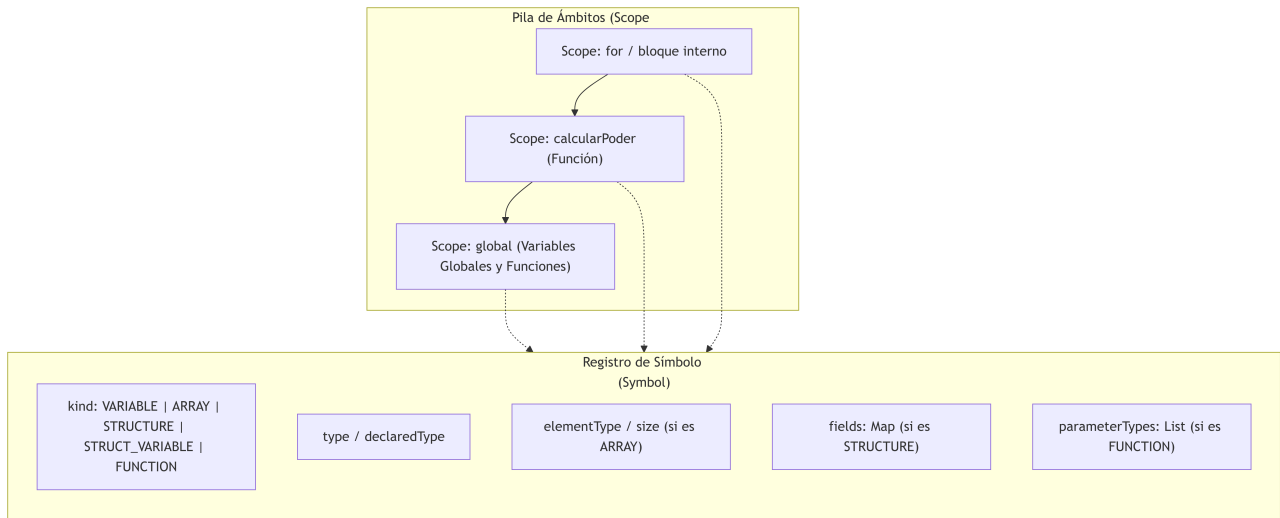
8.2. Matriz de Compatibilidad de Operaciones

Tipo Izquierdo	Operador	Tipo Derecho	Tipo Resultante	Regla Aplicada
numerus	+, -, *, /	numerus	numerus	Aritmética entera
numerus	+, -, *, /	decimalis	decimalis	Promoción a jerarquía superior (4 ¿ 3)
decimalis	+, -, *, /	numerus	decimalis	Promoción a jerarquía superior (4 ¿ 3)
decimalis	+, -, *, /	decimalis	decimalis	Aritmética punto flotante
littera	+, -, *, /	numerus	numerus	Promoción de carácter a entero (3 ¿ 2)
textum	+	Cualquier Tipo	textum	Concatenación: textum solo admite suma
Cualquier Tipo	+	textum	textum	Concatenación: textum solo admite suma
textum	-, *, /	Cualquiera	ERROR	textum no admite operaciones aritméticas
bool	&&,	bool	bool	Operación lógica booleana
Numérico	!, ¿, !=, ¿=	Numérico	bool	Comparación relacional numérica
Tipo A	==, !=	Tipo B	bool	Válido si $A.\text{accepts}(B)$ o $B.\text{accepts}(A)$
non (Unario)	-	bool	bool	Negación lógica estricta
- (Unario)	-	numerus/decimalis/littera	Mismo Tipo	Negación aritmética

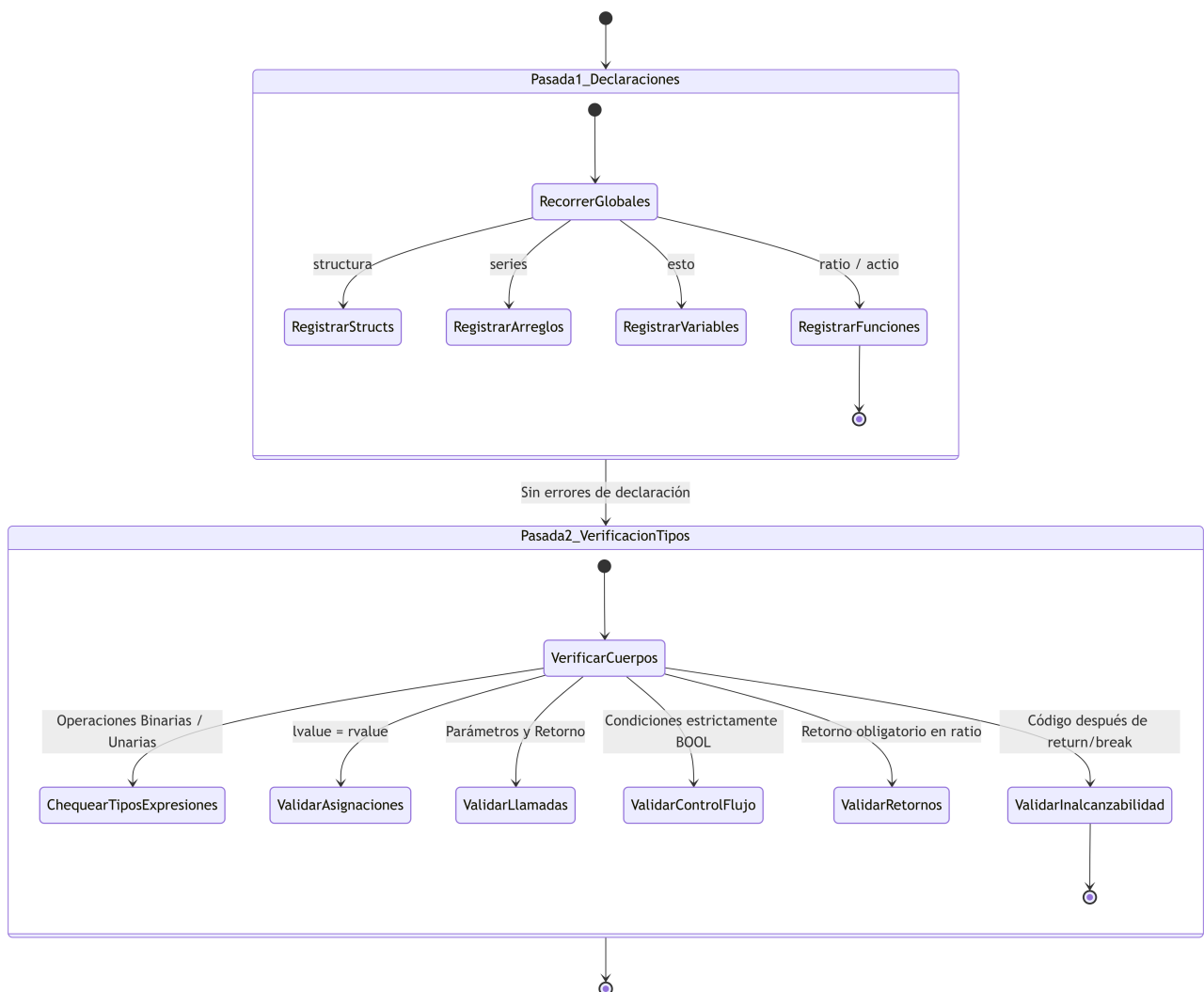
Cuadro 4: Matriz de compatibilidad de operaciones y reglas de inferencia.

8.3. Gestión de la Tabla de Símbolos (SymbolTable)

La tabla de símbolos maneja una pila de ámbitos (Deque<Scope>) que permite resolución de identificadores con sombreado de variables (*shadowing*).



8.4. Estrategia de Análisis Semántico en Dos Pasadas



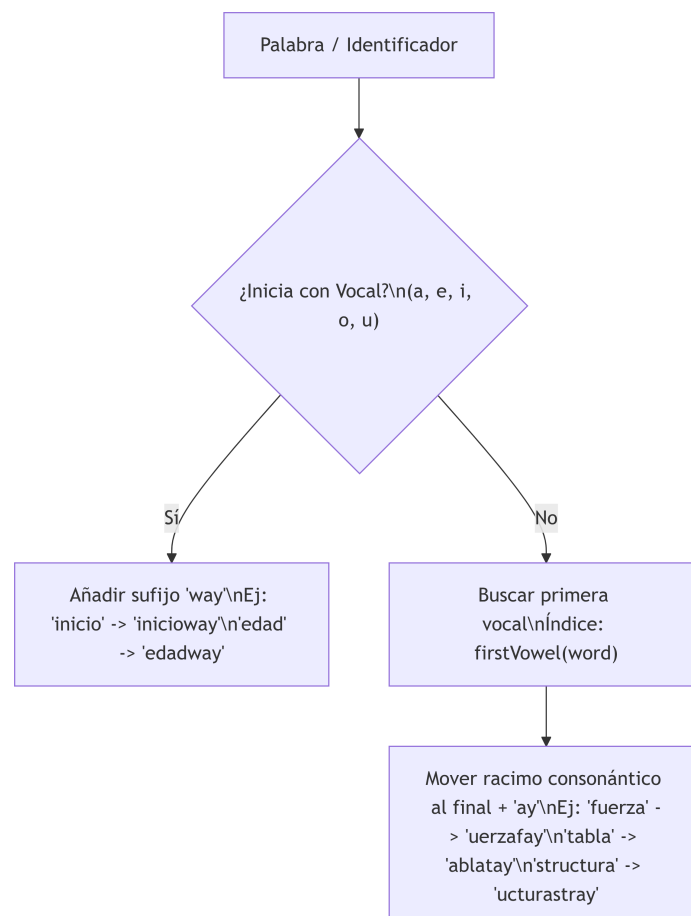
8.5. Validaciones Semánticas Específicas

- **Corrupción de Flujo:** Condiciones en sentencias `si`, `aliter`, `dum`, `facere...dum` y `per` deben evaluar estrictamente a `BOOL`. No se permite coerción numérica a booleano.
- **Retorno Exhaustivo en Funciones `ratio`:** Se valida que toda función `ratio` retorne un valor compatible en todos los caminos de ejecución posibles.
- **Código Inalcanzable:** Sentencias ubicadas después de un `reddere`, `interrumpe` o `perge` en el mismo bloque son marcadas como error de código no alcanzable.
- **Validación Estricta de Estructuras:** Se valida que no existan atributos duplicados en `structura`, que todos los campos requeridos se inicialicen en `structLiteral`, y que los tipos asignados a cada campo coincidan.
- **Evaluación Estática de Dimensiones e Índices:** Mediante `ConstantFolder`, el tamaño de arreglos y los accesos con índices constantes son evaluados en tiempo de compilación para reportar desbordamientos fuera de rango ($\text{index} < 0$ o $\text{index} \geq \text{size}$).

9. Motor de Traducción a Pig Latin

La traducción se ejecuta sobre el AST validado semánticamente (no mediante regex ni sustituciones textuales simples), recorriendo cada nodo en `PigLatinTranslator` y reconstruyendo el código con formato e indentación correctos.

9.1. Reglas de Transformación Morfológica (`PigLatinWordRules`)



9.2. Ley Porcina (Funciones de Entrada / Salida)

- Sentencia de impresión `>> expr`: Se traduce a `%OINK expr`; (o encadenado con `%OINK`).
- Sentencia de lectura `<< / id <<`: Se traduce a `%OINK_OINK / %OINK_OINK id_traducido`.

9.3. Tabla Comparativa de Traducción

Código Original (Codex Latinus)	Código Traducido (Pig Latin)
<code>esto edad : numerus 20;</code>	<code>estoway edadway : umerusnay 20;</code>
<code>esto cifrado : falsus;</code>	<code>estoway ifadrocay : alsusfay;</code>
<code>ratio numerus calcular(esto x : numerus)</code>	<code>atioray umerusnay alcularcay(estoway xway : umerusnay)</code>
<code>>> "Hola Comandante";</code>	<code>%OINK "Hola Comandante";</code>
<code>comandante <<</code>	<code>%OINK_OINK omandantecaysi</code>
<code>si (edad >= 18) { ... } finis;</code>	<code>isay (edadway >= 18) { ... } inisfay;</code>

Cuadro 5: Tabla comparativa de traducción Codex Latinus a Pig Latin.

10. Sistema Centralizado de Errores

10.1. Modelo CompilerError

Representa de forma inmutable cada anomalía encontrada durante la compilación:

- **fase**: Fase de detección (`Fase.LEXICA`, `Fase.SINTACTICA`, `Fase.SEMANTICA`).
- **mensaje**: Descripción detallada del error.
- **linea**: Número de línea en el archivo fuente (1-indexed).
- **columna**: Posición del carácter dentro de la línea.

10.2. Integración con ANTLR mediante CodexErrorListener

Los errores sintácticos y léxicos estándar emitidos por ANTLR son interceptados mediante `CodexErrorListener` (que implementa `BaseErrorListener`), impidiendo que salgan por `System.err` y registrándolos directamente en el `ErrorReporter` del pipeline.

10.3. Navegación Interactiva desde la GUI

Al hacer clic o doble clic en cualquier fila de la tabla de errores en `ResultsPanel`, el editor `EditorPanel` mueve automáticamente el cursor a la línea y columna exactas del error, facilitando la depuración al usuario.

11. Interfaz Gráfica de Usuario (IDE Swing)

La interfaz gráfica principal `MainFrame` está organizada en cuatro cuadrantes mediante `JSplitPane` anidados:

Editor de Código (.lat)	Salida / Traducción Pig Latin
- RSyntaxTextArea - Numeración de línea - Coloreado Codex Latinus	- RSyntaxTextArea (Solo Lectura) - Coloreado Pig Latin - Botón Exportar .pig
Árbol AST	Paneles de Diagnóstico (Tabs/Grid)
- Renderizado PNG con Graphviz - Zoom interactivo (+, -, reset) - Modal independiente de pantalla	4. Símbolos (JTable) 5. Pila ANTLR (Paso a paso / Todo) 6. Errores (JTable con navegación)

Cuadro 6: Estructura de cuadrantes de la Interfaz Gráfica.

11.1. Componentes Destacados

- **EditorPanel:** Editor principal configurado con `CodexLatinusTokenMaker` que mapea tokens de ANTLR directamente a estilos de `RSyntaxTextArea` con tema oscuro `MonokaiSyntaxTheme`.
- **ResultsPanel:** Panel de resultados multifuncional con soporte para visualización de imagen AST con zoom fluido (desde 25 % hasta 400 %), tabla de símbolos, visor de pila interactivo y tabla de errores.
- **CompilationPipeline Asíncrono:** La compilación se ejecuta en un hilo secundario mediante `SwingWorker`, manteniendo la interfaz responsiva en todo momento.
- **FileManager:** Manejo completo de archivos para abrir y guardar programas .lat y exportar archivos traducidos .pig.

12. Manual de Compilación, Ejecución y Pruebas

12.1. Requisitos Previos

- **Java Development Kit (JDK):** Versión 21 o superior configurada en `JAVA_HOME`.
- **Apache Maven:** Versión 3.8+ (o utilizar el wrapper `./mvnw` incluido).
- **Graphviz:** Binario `dot` accesible desde el `PATH` del sistema operativo (necesario para la generación de imágenes PNG del AST).

12.2. Compilación del Proyecto

Para compilar las clases del proyecto y procesar la gramática ANTLR:

```
./mvnw clean compile
```

12.3. Ejecución de la Aplicación

Para iniciar el IDE gráfico:

```
./mvnw exec:java
```

12.4. Ejecución de la Suite de Pruebas Unitarias

El proyecto cuenta con un conjunto de pruebas automatizadas que verifican:

- **TypeCheckingTests:** Pruebas de compatibilidad, coerción y errores de tipos.
- **PigLatinTranslationRulesTest:** Pruebas de reglas morfológicas de Pig Latin.
- **AstDotExporterTests:** Verificación de generación correcta de código DOT.

- **BooleanGrammarCheck:** Pruebas sintácticas de formas cortas y expresiones booleanas.
- **RegressionTest:** Pruebas de integración de extremo a extremo.

Para ejecutar todas las pruebas:

```
./mvnw test
```